

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Homework Assignment 1

- Will be released today.
- Due: **11:59 PM** on **Wednesday, February 11**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Do not upload the data file.
 - Make sure to **rename your notebook file** as instructed.

Lecture 5

- Data Wrangling:
 - Data Preparation and Cleaning
 - Inspecting and modifying data types (column)
 - String methods and method chaining
 - Handling missing values: filtering, imputation
 - Groupby and Split-Apply-Combine

About the data

- The dataset `spending_clean_ex.csv` was specifically constructed from the Medicare dataset.
- It was modified to contain interesting cleaning use cases.

```

spending_df = pd.read_csv('data/spending_clean_ex.csv', index_col='unique_id')
spending_df.head(7)

```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	\$77.26
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	\$767.83
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	\$5,409.29
TR390895	1639597115	NaN	LOSARTAN POTASSIUM	11.0	\$65.62
JA436080	1073781571	NaN	LAMOTRIGINE	12.0	\$8,873.70
WO822855	1689080749	INTERVENTIONAL CARDIOLOGY	SERTRALINE HCL	26.0	NaN
PC409862	1568483196	INTERNAL MEDICINE	CLOTRIMAZOLE/BETAMETHASONE DIP	12.0	\$310.97

Inspecting and modifying data types

- Recall that `dtypes` returns data type of each column.

```
spending_df.dtypes
```

```

doctor_id      int64
specialty      object
medication     object
nb_beneficiaries float64
spending       object
dtype: object

```

- Issues:

- `doctor_id` is saved as an `int64`.
- `spending` is saved as an `object`.

Inspecting and modifying data types (cont'd.)

- We need to cast the entries of each misidentified column to its appropriate Pandas data type.

```
spending_df['doctor_id'].astype('object')
```

- As is the case with most operations in Pandas, `astype()` is not in-place.

- We need to manually update the data to make the change permanent by overwriting the column.

```
spending_df['doctor_id'].astype('object')
spending_df.dtypes
```

```
doctor_id      int64
specialty      object
medication     object
nb_beneficiaries float64
spending       object
dtype: object
```

Changing column data type

```
spending_df['doctor_id'] = spending_df['doctor_id'].astype('object')
```

```
spending_df.dtypes
```

```
doctor_id      object
specialty      object
medication     object
nb_beneficiaries float64
spending       object
dtype: object
```

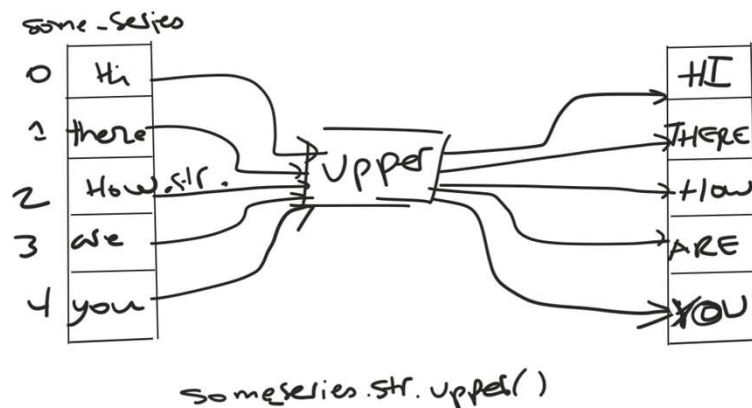
String methods on Series of type object

- A Series with string-like values provides the `.str` accessor.
`spending_df["spending"].str`
- The `.str` accessor exposes a collection of **vectorized string methods**.
 - These methods are the same as those available on Python str objects (e.g., `upper()`, `contains()`, `split()`, etc.).
 - Unlike Python string methods, they are applied element-wise to each entry in the Series.
- The result of a `.str` operation is a new Series.
- String methods are not applied directly to the Series itself, but through the `.str` attribute.

String methods on Series of type object

```
some_series = pd.Series(["Hi", "there", "How", "are", "you"])
some_series.str.upper()
```

```
0      HI
1    THERE
2     HOW
3     ARE
4     YOU
dtype: object
```



Using replace() to fix column's data

- We need to remove the “\$” character.
 - With replace(), we can remove “\$” by replacing it with the "" (empty string).

```
spending_df["spending"].str.replace("$", "")
```

- We need to remove the “,” character (ex. 3,454,420.29 should be 3454420.29).
 - With replace(), we can remove “,” by replacing it with the "" (empty string).
- Since changes are not permanent, we need to overwrite the data in the column.

```
spending_df.sort_values(by="spending").head(10)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
CQ964168	1174510481	OBSTETRICS/GYNECOLOGY	NYSTATIN/TRIAMCIN	11.0	\$1,219.38
BU202944	1538156542	FAMILY PRACTICE	ALBUTEROL SULFATE	16.0	\$1,456.08
RH274400	1023342144	NURSE PRACTITIONER	DICLOFENAC SODIUM	32.0	\$1,491.24
MK361461	1245206184	CARDIOLOGY	PANTOPRAZOLE SODIUM	102.0	\$1,608.42
ON964391	1528031978	INTERNAL MEDICINE	FENTANYL	NaN	\$1,730.84
MG439448	1962498311	OPHTHALMOLOGY	PILOCARPINE HCL	20.0	\$1,910.22
XI271888	1982785085	INTERNAL MEDICINE	BUDESONIDE/FORMOTEROL FUMARATE	37.0	\$10,253.35
DJ966828	1629022389	NURSE PRACTITIONER	EZETIMIBE	20.0	\$10,869.06
SU660670	1366528028	INTERNAL MEDICINE	LISINOPRIL	25.0	\$118.89
DA943932	1659338887	PAIN MANAGEMENT	OXYCODONE HCL	52.0	\$15,272.18

```

spending_df["spending"] = spending_df["spending"].str.replace("$", "")
spending_df["spending"] = spending_df["spending"].str.replace(",", "")
spending_df["spending"] = spending_df["spending"].astype('float64')
spending_df.head()

```

'float'

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	77.26
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	767.83
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	5409.29
TR390895	1639597115	NaN	LOSARTAN POTASSIUM	11.0	65.62
JA436080	1073781571	NaN	LAMOTRIGINE	12.0	8873.70

spending_df.dtypes

```

doctor_id      object
specialty      object
medication     object
nb_beneficiaries float64
spending       float64
dtype: object

```

Combining multiple string operations

- We can use method chaining to carry out the operation.
 - Combines similar functionality in one line.
 - Easier to understand and maintain.
- Note that we need to call the 'str' of each resulting Series.

```

spending_df["spending"] = (spending_df["spending"]
    .str.replace("$", "")
    .str.replace(",", "")
    .astype("float64"))

```

*.str.replace(["\$", ","], "") ⇔ { .str.replace("\$", "")
 " ") .astype("float64") }*

Combining multiple string operations (cont'd.)

```
spending_df = pd.read_csv('data/spending_clean_ex.csv', index_col='unique_id')

spending_df["spending"] = (spending_df["spending"]
                           .str.replace("$", "")
                           .str.replace(",", "")
                           .astype("float64"))

spending_df.dtypes
```

```
doctor_id          int64
specialty          object
medication         object
nb_beneficiaries   float64
spending           float64
dtype: object
```

Missing data

- Missing data can complicate analysis or lead to erroneous results.
- Missing data can arise for various reasons.
 - Human error: Data deleted by mistake.
 - Partial data collection.
 - E.g., the first round of data collection did not record a field (ex. salary) while the second round did.
 - Defective instrumentation or intermittent communication problems.
- Missing values can be indicated using a sentinel.
 - E.g., 999999, “UNKNOWN”, “N/A”, etc.

Handling missing data

- Requires identifying missing values.
- Handling missing values is typically done by either:
 1. Filtering out missing values.
 2. Filling in missing values with putative values (imputation).
- When to use each option is application-specific and data-specific.
 - Can you afford to throw away data?
 - Is there an easy way to fill in, i.e., impute, missing values?
 - Use default value for instance (ex. mean for that feature if numeric value).
 - Use a statistical model.
 - ...

Identifying missing values

- Missing value can occur in the data as:
 - Empty fields indicating that the value is missing.
 - Pandas denotes missing values with NaN.
 - Automatically set if the value is empty in the data file.
 - NaN available in numpy is represented as a float that is not equal to any other number. (`np.nan != np.nan`)
`np.nan == np.nan`
- Sentinel values
`False`
 - Sentinels are typically dummy data that is added convenience to identify that a value is missing.
 - Ex. replacing zip code with 999999 or salary with 0 or negative value (ex. -1).

Identifying missing values (cont'd.)

- Identifying missing values is critical for appropriately modeling the data.
 - Leaving sentinel value in can skew the data.
 - For example, computing mean salary if data contains sentinel value will skew the mean.
 - Leaving NaN values can lead to NaN as a result.
 - For example, replace all the occurrences of 999999 with `np.nan`.

Identifying missing values (cont'd.)

- We can use the `isnull()` or `isna()` methods on a Series or DataFrame to identify missing values.
 - The method returns True if value is NaN and False otherwise.

```
spending_df.head()
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	77.26
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	767.83
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	5409.29
TR390895	1639597115	NaN	LOSARTAN POTASSIUM	11.0	65.62
JA436080	1073781571	NaN	LAMOTRIGINE	12.0	8873.70

```
spending_df.isnull().head()
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	False	False	False	False	False
CG916968	False	False	False	False	False
SA964720	False	False	False	False	False
TR390895	False	True	False	False	False
JA436080	False	True	False	False	False

```
spending_df.isna().head()
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	False	False	False	False	False
CG916968	False	False	False	False	False
SA964720	False	False	False	False	False
TR390895	False	True	False	False	False
JA436080	False	True	False	False	False

Counting missing values

- You can count the number of missing values by counting the number of True and False values.

- Recall that Python encodes True as 1 and False as 0.
 - Use sum to count the number of 0s and 1s.

```
spending_df["spending"].isnull().sum()
```

- You can achieve the same by subtracting the result of count() (returns the number of non-NA/null) method from the result of the size attribute (including missing values).

```
spending_df["spending"].size - spending_df["spending"].count()
```

```
spending_df['spending'].isnull().sum()
```

4

```
print(spending_df["spending"].size - spending_df["spending"].count())
```

4

```
spending_df.isnull().sum()
```

doctor_id	0
specialty	6
medication	1
nb_beneficiaries	4
spending	4
dtype: int64	

```
spending_df.count()
```

doctor_id	54
specialty	48
medication	53
nb_beneficiaries	50
spending	50
dtype: int64	

Changing column data type

- Converting as we did earlier can only be done if the data in the column is compatible with the type.
 - E.g., we cannot convert a column that contains last names into float64 (number with decimal points).
 - Similarly, we cannot convert a column that contains a NaN to integer (recall that NaN is float).

```
x = pd.DataFrame([[1, "10"], [2, np.nan], [3, "4"]])
x[1].astype('int')
...
ValueError: cannot convert float NaN to integer
```
- Converting a column that contains a NaN to str results in NaN becoming the string "nan".

```
x = pd.DataFrame([[1, "10"], [2, np.nan], [3, "4"]])
display(x)

print(x.dtypes)
print("-" * 10)
x[1].astype('float64')
```

```

  0    1
0  1   10
1  2  NaN
2  3    4

0      int64
1      object
dtype: object
-----
0      10.0
1      NaN
2       4.0
Name: 1, dtype: float64
```

```
y = x[1].astype('str')

display(y)
print(y[1], type(y[1]))

0      10
1      nan
2       4
Name: 1, dtype: object
nan <class 'str'>
```

Changing column data type (cont'd.)

- `nb_beneficiaries` cannot readily be converted to `int64`.

```
spending_df['nb_beneficiaries'] = spending_df['nb_beneficiaries'].astype('int64')
...
```

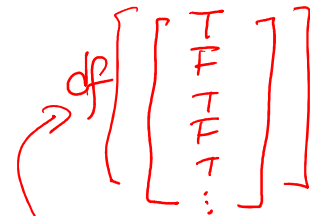
IntCastingNaNError: Cannot convert non-finite values (NA or inf) to integer

- Pandas cannot convert `NaN` to `int`.
 - We need to first handle the missing value before converting.

Filtering missing values

- The trivial way to handle missing values is by dropping entries that contain them.
- Judgement call as to whether this can bias your analysis.
 - For instance, removing 5 instances out of 1M that have missing values should not be a problem.
 - However, if all males have salary value missing, removing all males' data will bias your market segmentation model.
- Can be carried out using subsetting or the `dropna()` method.

Filtering missing values (cont'd.)



- You **can** subset using a Boolean Series.
 - We previously saw how to subset using a list of indices and labels:

```
null_entries = spending_df["spending"].isnull() → Boolean Series  
not_null_entries = ~null_entries  
spending_no_na_df = spending_df[not_null_entries]
```

or simply

```
spending_no_na_df = spending_df[~spending_df["spending"].isnull()]
```

- Boolean masks must match the length of the axis being indexed.

```
data = pd.DataFrame({"A": [1, 2, 3, 4, 5, 6], "B": [10, 2, 5, 7, np.nan, 19]})
data
```

	A	B
0	1	10.0
1	2	2.0
2	3	5.0
3	4	7.0
4	5	NaN
5	6	19.0

~is_null_values

```
is_null_values = data["B"].isnull()
is_null_values
```

```
0    False
1    False
2    False
3    False
4     True
5    False
Name: B, dtype: bool
```

```
data[~is_null_values]
```

	A	B
0	1	10.0
1	2	2.0
2	3	5.0
3	4	7.0
5	6	19.0

```
null_entries = spending_df["spending"].isnull()
not_null_entries = ~null_entries
spending_no_na_df = spending_df[not_null_entries]

print(spending_df.shape, spending_no_na_df.shape)
```

```
(54, 5) (50, 5)
```

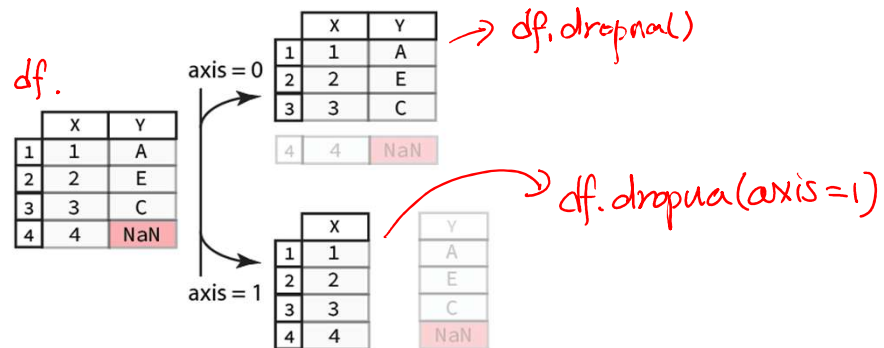
```
spending_no_na_df = spending_df[~spending_df["spending"].isnull()]

print(spending_df.shape, spending_no_na_df.shape)
```

```
(54, 5) (50, 5)
```

Filtering NaN using the dropna() method

- Considers, by default, all the columns.
 - Can consider a subset of columns.
- The method can be used to drop either rows or columns using axis.



```
test_df = pd.DataFrame({"X": [1, 2, 3, 4], "Y": ['A', 'E', 'C', np.nan]})
test_df
```

	X	Y
0	1	A
1	2	E
2	3	C
3	4	NaN

```
test_df.dropna()
```

	X	Y
0	1	A
1	2	E
2	3	C

```
test_df.dropna(axis=1)
```

	X
0	1
1	2
2	3
3	4

```
# dropna() is not in-place operation
test_df
```

	X	Y
0	1	A
1	2	E
2	3	C
3	4	NaN


```
test_df.dropna(axis=0, subset=["X"])
```

	X	Y
0	1	A
1	2	E
2	3	C
3	4	NaN

```
test_df.dropna(axis=0, subset=["Y"])
```

	X	Y
0	1	A
1	2	E
2	3	C

Filling missing values

- Filling missing values is often referred to as *missing value imputation*, or simply *imputation*.
- When data is scarce, being able to impute missing values is critical.
- Two high-level strategies for imputing data:
 1. Filling the value with a representative constant.
 2. Filling the value dynamically.
 - How many cars went through the University Ave's H1 exit this Wednesday?
 - Probably close to the same number as last Wednesday.
 - What was the temperature reported by a sensor at 2 PM?
 - An average of the temperatures at times 1:55 PM and 2:05 PM is probably a good estimate.
- Both approaches can be carried out using `fillna()`.

fillna() with static values

- Use a scalar constant to replace all missing values.

```
filled_spending_df = spending_df.fillna(0)
```

- Use a dictionary with specific values for each column.

```
filled_spending_df = spending_df.fillna(  
    {"specialty": "UNKNOWN",  
     "nb_beneficiaries": 0,  
     "spending": 0}  
)
```

fillna() with dynamically computed values

- Dynamic value imputation refers to filling the missing entries with a value that depends on the existing data.
 - Can reflect the patterns in the dataset
 - Often preferred when sufficient data is available
- Many different approaches for doing this.
 - Ex. replace a value with the column's mean.

```
average_spending = spending_df["spending"].mean()  
average_nb_beneficiaries = spending_df["nb_beneficiaries"].mean()  
filled_spending_df = spending_df.fillna( {"nb_beneficiaries": average_nb_beneficiaries,  
                                           "spending": average_spending} )
```

spending_df

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	77.26
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	767.83
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	5409.29
TR390895	1639597115	NaN	LOSARTAN POTASSIUM	11.0	65.62
JA436080	1073781571	NaN	LAMOTRIGINE	12.0	8873.70
WO822855	1689080749	INTERVENTIONAL CARDIOLOGY	SERTRALINE HCL	26.0	NaN
PC409862	1568483196	INTERNAL MEDICINE	CLOTTRIMAZOLE/BETAMETHASONE DIP	12.0	310.97
CG865025	1497955603	CARDIOLOGY	ATORVASTATIN CALCIUM	82.0	2726.72
XI271888	1982785085	INTERNAL MEDICINE	BUDESONIDE/FORMOTEROL FUMARATE	37.0	10253.35
EQ932492	1336217306	INTERNAL MEDICINE	OXYCODONE HCL	NaN	355.43
DT789371	1518992585	NaN	FENOFIBRATE NANOCRYSTALLIZED	33.0	2565.44
:					

```
filled_spending_df = spending_df.fillna(0)
filled_spending_df
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	77.26
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	767.83
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	5409.29
TR390895	1639597115	0	LOSARTAN POTASSIUM	11.0	65.62
JA436080	1073781571	0	LAMOTRIGINE	12.0	8873.70
WO822855	1689080749	INTERVENTIONAL CARDIOLOGY	SERTRALINE HCL	26.0	0.00
PC409862	1568483196	INTERNAL MEDICINE	CLOTTRIMAZOLE/BETAMETHASONE DIP	12.0	310.97
CG865025	1497955603	CARDIOLOGY	ATORVASTATIN CALCIUM	82.0	2726.72
XI271888	1982785085	INTERNAL MEDICINE	BUDESONIDE/FORMOTEROL FUMARATE	37.0	10253.35
EQ932492	1336217306	INTERNAL MEDICINE	OXYCODONE HCL	0.0	355.43
DT789371	1518992585	0	FENOFIBRATE NANOCRYSTALLIZED	33.0	2565.44
:					

```
filled_spending_df = spending_df.fillna(
    {"specialty": "UNKNOWN",
     "nb_beneficiaries": 0,
     "spending": 0}
)
filled_spending_df.head(10)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	77.26
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	767.83
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	5409.29
TR390895	1639597115	UNKNOWN	LOSARTAN POTASSIUM	11.0	65.62
JA436080	1073781571	UNKNOWN	LAMOTRIGINE	12.0	8873.70
WO822855	1689080749	INTERVENTIONAL CARDIOLOGY	SERTRALINE HCL	26.0	0.00
PC409862	1568483196	INTERNAL MEDICINE	CLOTRIMAZOLE/BETAMETHASONE DIP	12.0	310.97
CG865025	1497955603	CARDIOLOGY	ATORVASTATIN CALCIUM	82.0	2726.72
XI271888	1982785085	INTERNAL MEDICINE	BUDESONIDE/FORMOTEROL FUMARATE	37.0	10253.35
EQ932492	1336217306	INTERNAL MEDICINE	OXYCODONE HCL	0.0	355.43
⋮					

```
average_spending = spending_df["spending"].mean()
average_nb_beneficiaries = spending_df["nb_beneficiaries"].mean()
filled_spending_df = spending_df.fillna({"nb_beneficiaries": average_nb_beneficiaries,
                                         "spending": average_spending})
filled_spending_df
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	77.260
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	767.830
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	5409.290
TR390895	1639597115	NaN	LOSARTAN POTASSIUM	11.0	65.620
JA436080	1073781571	NaN	LAMOTRIGINE	12.0	8873.700
WO822855	1689080749	INTERVENTIONAL CARDIOLOGY	SERTRALINE HCL	26.0	5806.981
PC409862	1568483196	INTERNAL MEDICINE	CLOTRIMAZOLE/BETAMETHASONE DIP	12.0	310.970
CG865025	1497955603	CARDIOLOGY	ATORVASTATIN CALCIUM	82.0	2726.720
XI271888	1982785085	INTERNAL MEDICINE	BUDESONIDE/FORMOTEROL FUMARATE	37.0	10253.350
EQ932492	1336217306	INTERNAL MEDICINE	OXYCODONE HCL	55.6	355.430
DT789371	1518992585	NaN	FENOFIBRATE NANOCRYSTALLIZED	33.0	2565.440

Forward and backward filling missing values

- `bfill()` method: backward fill
 - Fills a missing value with the next non-missing value that comes **after** it.
- `ffill()` method: forward fill
 - Fills a missing value with the previous non-missing value that **comes** before it.
- Another strategy is to just average the previous and next values.

```
complete_data = pd.Series([10, 20, 20, 5, 30, 100, 30, 20, 10, 10,
                           20, 10, 5, 20, 50, 40, 50, 50, 50, 50, 60])
test_df = pd.Series([10, 20, 20, 5, np.nan, 100, 30, 20, 10, 10,
                    20, 10, 5, np.nan, 50, 40, 50, 50, 50, 50, 60])
data = pd.DataFrame({"Complete": complete_data, "test_df": test_df})
```

	Complete	test_df
0	10	10.0
1	20	20.0
2	20	20.0
3	5	5.0
4	30	NaN
5	100	100.0
6	30	30.0
7	20	20.0
8	10	10.0
9	10	10.0
10	20	20.0
11	10	10.0
12	5	5.0
13	20	NaN
14	50	50.0
15	40	40.0
16	50	50.0
17	50	50.0
18	50	50.0
19	50	50.0
20	60	60.0

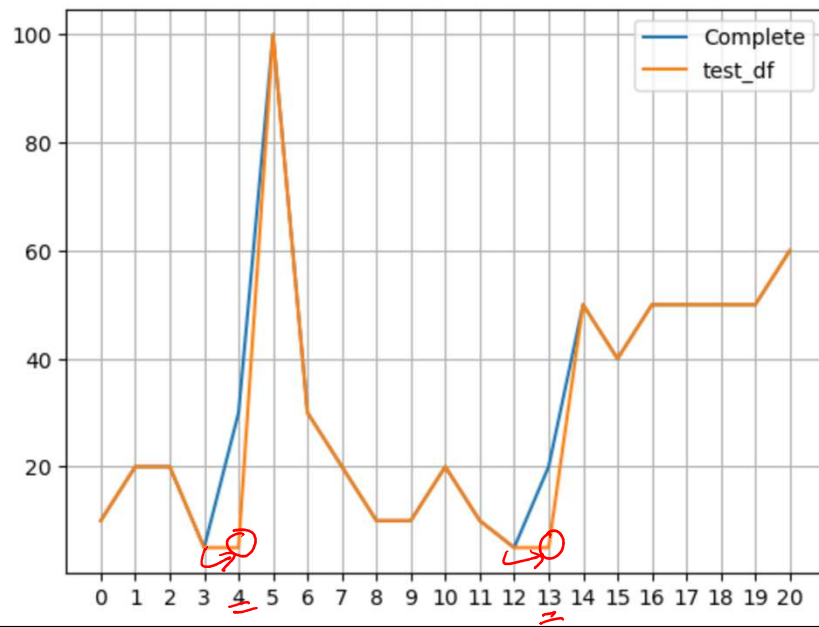
```
ffilled_data = data.ffill()  
ffilled_data
```

	Complete	test_df
0	10	10.0
1	20	20.0
2	20	20.0
3	5	5.0
4	30	5.0
5	100	100.0
6	30	30.0
7	20	20.0
8	10	10.0
9	10	10.0
10	20	20.0
11	10	10.0
12	5	5.0
13	20	5.0
14	50	50.0
15	40	40.0
16	50	50.0
17	50	50.0
18	50	50.0
19	50	50.0
20	60	60.0

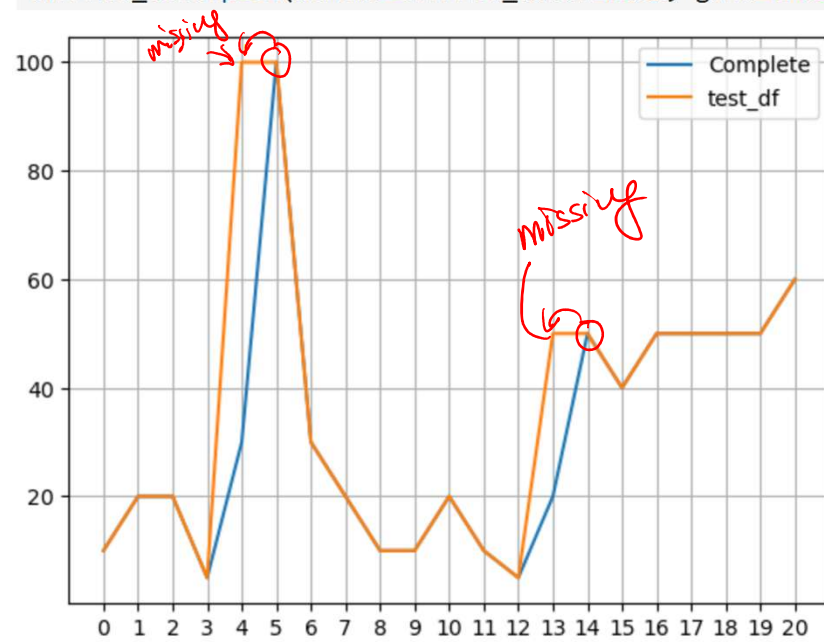
```
bfilled_data = data.bfill()  
bfilled_data
```

	Complete	test_df
0	10	10.0
1	20	20.0
2	20	20.0
3	5	5.0
4	30	100.0
5	100	100.0
6	30	30.0
7	20	20.0
8	10	10.0
9	10	10.0
10	20	20.0
11	10	10.0
12	5	5.0
13	20	50.0
14	50	50.0
15	40	40.0
16	50	50.0
17	50	50.0
18	50	50.0
19	50	50.0
20	60	60.0

```
ffilled_data.plot(xticks=ffilled_data.index, grid=True)
```



```
bffilled_data.plot(xticks=bffilled_data.index, grid=True)
```



Filling with dynamically computed values (cont'd.)

- Dozens of possible strategies.
 - We will see regression as a simple model to impute missing values.
 - More sophisticated machine learning models can be used.
- When instances (entries) are similar, a good strategy is to just copy value from previous or next instance.
 - Alternatively, we can average based on k-nearest neighbors.
- You can come up with your own model (logic) and apply it.

Exercise: Data preparation and Cleaning

- The file `exercise_data_clean_prep.ipynb` is available on Lamaku. Please download it and complete the exercises.
- No submission required. Discuss the open-ended questions in Slack to earn class participation credit.